Clase 16 · Del servidor secuencial al concurrente con fork()

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · martes 20 oct 2026 · Tema 2 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. Repaso: Programa 05, servidor secuencial

Servidor TCP que atiende a los clientes **de uno en uno**, en orden de llegada; los demás esperan en la cola de `listen` (Clase 13).

1. `while(1)` externo con `accept()`.
2. Al aceptar, `do-while` interno de conversación síncrona: `recv()` la petición, la respuesta se **teclea en el servidor** (stdin) y se manda con `send()`.
3. Excepción: si el cliente manda **"adiós"**, un `if` carga en el buffer una **respuesta predeterminada** y la envía automáticamente.
4. Sale del `do-while`, `close(fd_c)` y vuelve a `accept()` por el siguiente del backlog.

Limitación: mientras un cliente conversa, los demás no avanzan (Tarea 4, `sk_acceptq_is_full`).
Error común: creer que la respuesta a "adiós" se teclea; es automática.

2. Programa 06: servidor concurrente con fork()

Atiende a varios clientes a la vez creando con `fork()` **un proceso hijo por conexión**. Archivos: `programa_06_servidor_inet.c` y `programa_06_cliente_inet.c`.

- El padre crea `fd_s`, hace `bind()` y `listen()`, y se bloquea en `accept()` dentro de `while(1)`.
- `accept()` devuelve `fd_c`; el padre llama a `fork()`: devuelve al **padre el PID del hijo (> 0)** y al **hijo 0** (Clases 02 y 04).

Hijo (`if (pid == 0)`)	**Padre**
Declara localmente `buff_peticion` y `buff_respuesta` (no gasta memoria en el padre)	—
`close(fd_s)`: nunca aceptará conexiones	`close(fd_c)`: se desentiende de esa conversación
Bucle: `memset`, `recv()`, imprime, responde (stdin o automática), `send()`	Regresa al `while(1)` y se bloquea en `accept()`
Al recibir "adiós": `close(fd_c)` y `exit(0)`	Atiende al siguiente cliente de inmediato

Resultado

El tamaño del backlog de `listen` deja de ser limitante crítica: las conexiones ya no esperan a que termine la anterior. Une procesos del Tema 1 (`fork()`, PID, PPID) con sockets del Tema 2 (subtema 2.3.1, servidor concurrente).

Error común: olvidar `close(fd_s)` en el hijo o `close(fd_c)` en el padre → **fuga de descriptores**.

3. Arquitecturas reales y escalabilidad

- Frontend (móvil/web) → **balanceador de carga** → clúster de servidores de aplicaciones (backend) → clústeres de bases de datos.
- Reparto por **Round Robin** (petición 1 al backend 1, 2 al 2…) o por **métricas** de CPU/memoria (pasar tráfico al secundario cuando el principal llega al 90 %).
- Caso de un alumno: frontends y backends por SSH, bases separadas (transaccionales y de usuarios). Respaldos: transaccionales cada 2–3 h (dinero), usuarios cada semana, generales cada mes.

Recuperación ante desastres

DRP (Disaster Recovery Plan) y regla **3-2-1**: 3 copias, en al menos 2 medios o ubicaciones, 1 fuera de la sede, en lugares con riesgos distintos (zona sísmica, costa con huracanes, montaña).

Error común: un clúster web potente con **una sola** base de datos central → cuello de botella (como la publicación de resultados de admisión UNAM).

4. Revisión del log del examen parcial

Log del servidor del profesor (132.248.59.6, puertos 4898/4899) con los exámenes del 15 al 19 de octubre.

- **Error del profesor:** buffer fijo de 50 bytes para fecha, hora, IP y nombre; la cadena real ocupaba 38–39 bytes y **no limpió con `memset` ni puso `'\0'`**: los 11 bytes sobrantes salieron como basura en el archivo.
- **Intrusiones:** por ser IP pública llegaron bots que escanean puertos. El 15 y 16 oct (19:10 y 19:11) el escáner **Infrawatch** (Londres) mandó peticiones con cabeceras HTTP para identificar el servicio; también hubo **fuerza bruta al puerto 22 (SSH)**.

Lección: todo servicio con IP pública recibe escaneos en horas; siempre limpiar y truncar los buffers (Clases 13 y 14).

Avisos

- Sin tarea. Tarea moral: compilar y probar el Programa 06 con varias terminales cliente.
- ACS-E01: quien no haya visto su confirmación de registro, que busque al profesor.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Servidor	Para recordar
Secuencial (P05)	Un cliente a la vez; los demás en la cola de `listen`; "adiós" → respuesta automática
Concurrente (P06)	`fork()` por cliente: el padre acepta, el hijo conversa

Cierre cruzado (examen)

Hijo: `close(fd_s)`, conversa con `fd_c`, `exit(0)` al "adiós". Padre: `close(fd_c)` y vuelve a `accept()`. `fork()` devuelve PID > 0 al padre y 0 al hijo. Buffers dentro del bloque del hijo.

Producción. Balanceador (Round Robin o métricas) → backends → clúster de BD; DRP y respaldos 3-2-1; una sola BD central es cuello de botella.

Log. Buffer sin `memset` ni `'\0'` → basura. IP pública → bots (Infrawatch) y fuerza bruta a SSH.